설비 고장을 예측하는 AI 데이터 분석

NumPy 실습

통계 기반 이상 탐지와 종합 분석

posco

CONTENTS

목차

빈 칸을 다루는 법부터 정비 판단까지

01 **결측 대응과 이상 탐지 기법** — isnan · nanmean · percentile · IQR · z-score · 시그마 규칙

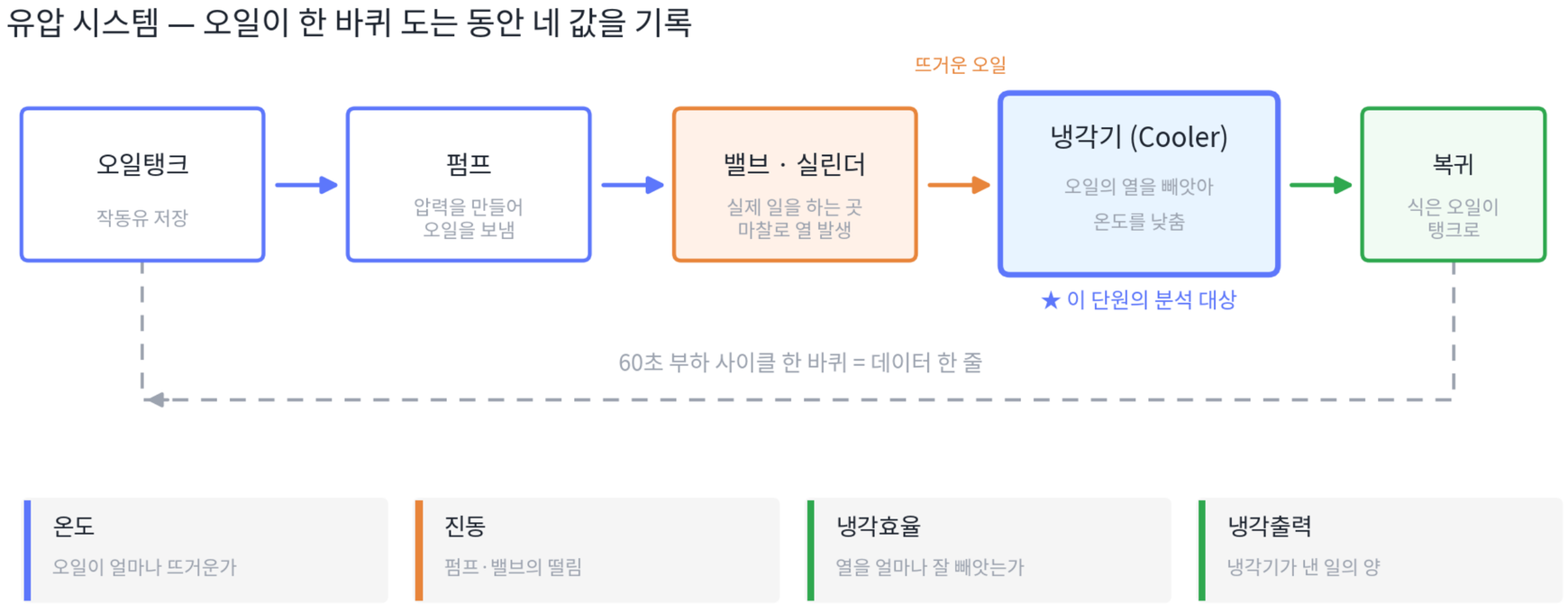
02 **다중 센서와 종합 분석** — corrcoef · axis별 통계 · 변화점 탐지 · 종합 이상 점수

03 **종합 이상 탐지 리포트** — 실습 8개 · 유압 냉각기 이상 탐지와 정비 판단

posco

분석 대상 설비 — 유압 시스템의 냉각기

펌프가 만든 압력으로 일을 하고, 뜨거워진 오일을 냉각기가 식혀 다시 돌려보냄



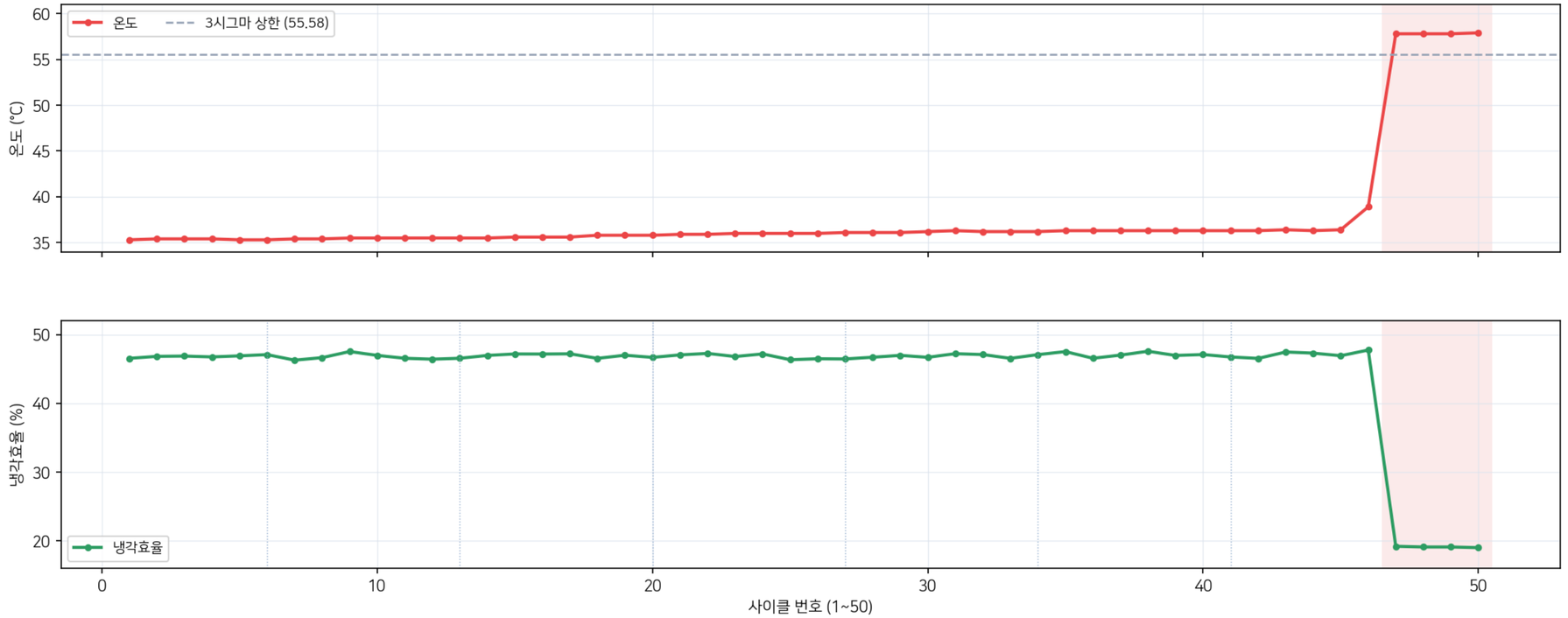
데이터의 한 행을 60초 운전 사이클 하나로 가정 · 전체 50사이클을 합성함

냉각기가 나빠지면 오일을 못 식혀 온도가 오르고 냉각효율이 떨어짐 — 오늘의 분석 대상

데이터: 설비 구성은 UCI 유압 시스템 자료 참고 · 수치는 교육용으로 생성한 합성 데이터(50사이클·6열)

정상 사이클과 냉각기 고장 임박 — 무엇이 다른가

정상 46사이클은 35도 부근에서 안정 · 고장 임박 4사이클은 57도대로 뛰어오름



고장 임박으로 설정한 구간에서 온도는 높고 냉각효율은 낮음 — 강한 음의 상관관계

세로 점선 여섯 개는 실습을 위해 의도적으로 만든 진동 결측 위치

상태 열은 교육용으로 부여한 라벨 · 정상 46 · 이상 4 — 실제 고장 판정 자료가 아님

posco

오늘의 목표 — 통계로 이상 센서 찾기

통계를 무기로 이상 센서를 자동으로 찾아내는 분석가의 기본기 완성

오늘은 통계를 무기로 이상 센서를 자동으로 찾아내는 시간

평균 · 표준편차 · 사분위수가 **이상 판정의 기준**

어제 익힌 손기술 위에 통계라는 판단 기준을 더하는 시간

posco

기초 통계 함수 — 어제의 도구

평균 · 중앙값 · 표준편차 · 분산 — 오늘 이상 탐지의 재료

01 · 평균 · 중앙값

한가운데를 가리키는 두 잣대

np.mean · np.median

mean은 한가운데, median은 줄 세운 정중앙. 극단값에 덜 흔들림

둘이 크게 다르면 **이상값 의심**

02 · 표준편차 · 분산

퍼짐의 정도

np.std · np.var

값들이 평균에서 얼마나 퍼져 있는지를 수치화

이상 탐지의 **핵심 재료**

03 · 최대 · 최소 · axis

2D 배열의 방향

np.max · np.min

2D에서 axis 옵션으로 행 · 열 방향을 정해 계산

축은 **없어지는 방향**

posco

Boolean indexing · where 복습

조건 필터링은 이상 탐지의 실행 도구 — 통계가 기준, 필터링이 실행

통계로 기준을 세우면, 마지막엔 **조건으로 골라냄**

STEP 1

비교 연산

True · False 배열 생성

STEP 2

불리언 인덱싱

조건이 True인 값만 추출

STEP 3

np.where

기준 넘으면 이상, 아니면 정상으로 라벨링

IQR이든 z-score든, 마지막엔 전부 조건으로 골라내는 단계로 끝남

posco

조건 결합과 위치 찾기

여러 조건을 묶거나, 만족하는 자리의 위치를 찾는 도구

01 · 그리고 조건

두 조건 모두 만족

```
(a > 70) & (a < 80)
```

앰퍼샌드로 연결, 각 조건을 괄호로 감싸는 것이 필수

괄호 빠지면 **계산 순서 오류**

02 · 또는 조건

둘 중 하나만 만족

```
(a < 70) | (a > 80)
```

막대 기호로 연결, IQR 상한 · 하한 둘 중 하나라도 벗어남에 활용

범위 **밖**을 잡을 때

03 · 위치 찾기

어디서 발생했나

```
np.where(조건)
```

조건만 주면 만족하는 자리의 인덱스 반환

이상 **발생 위치** 추적

posco

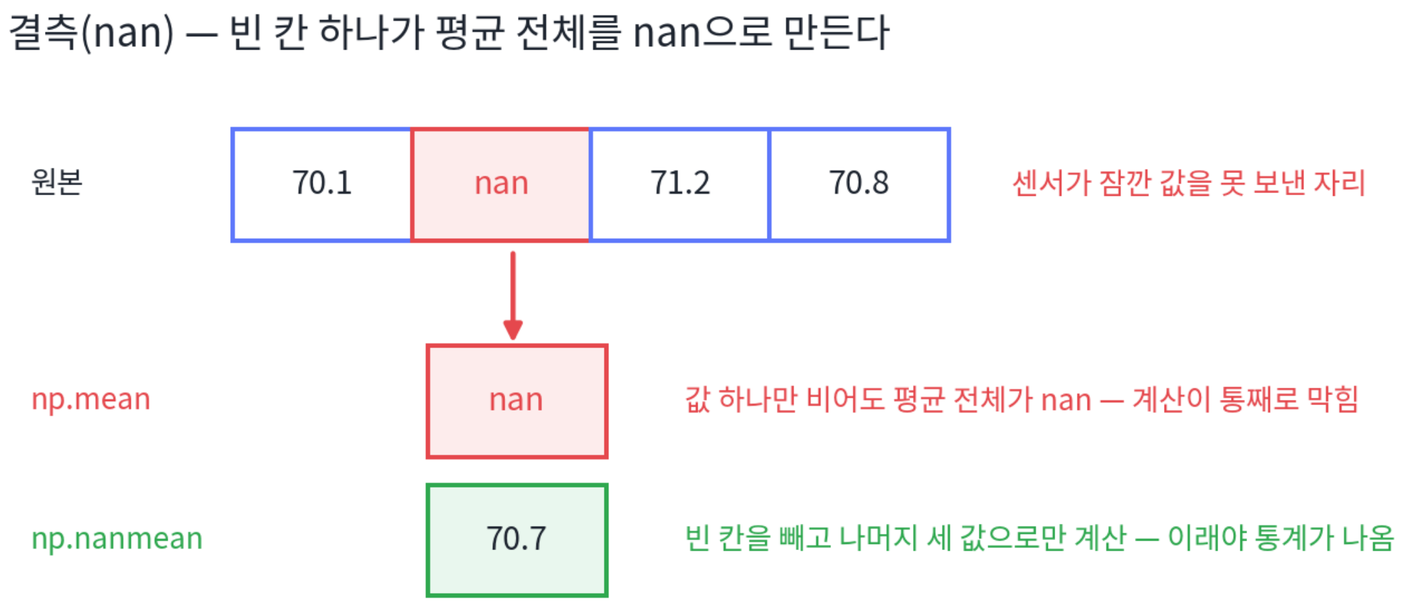
01

결측 대응과 이상 탐지 기법

isnan · nanmean · percentile · IQR · z-score · 시그마 규칙

결측값 위치 찾기 — isnan

NumPy는 값이 없는 자리를 nan으로 표시 — 통계의 가장 큰 골칫거리



nan은 자기 자신과도 같지 않아 등호로는 찾을 수 없음 — 전용 함수로 위치를 확인해야 함

값 하나만 비어 있어도 평균 전체가 nan이 되어 계산이 통째로 막힘

빈 칸은 「값이 0」이 아니라 「값을 모름」 — 0으로 채우면 평균이 아래로 끌려감

posco

isnan — 핵심 코드

결측 위치와 개수 찾기 — True · False 배열에 sum으로 개수 집계

CODE

```
# 결측이 섞인 온도 센서 배열
temp = np.array([70.1, np.nan, 71.2, np.nan])

# 결측 위치 (True가 nan 자리)
print(np.isnan(temp))
# 출력: [False True False True]

# 결측 개수
print(np.isnan(temp).sum())
# 출력: 2
```

np.where와 결합하면 위치 인덱스도 확보

posco

결측 무시 통계 — nanmean · nanstd

결측을 없는 셈 치고 나머지 값으로만 통계 계산

np.nanmean · np.nanstd — 결측을 건너뛰는 통계 함수

일반 mean은 nan에 막혀 nan을 반환, nanmean은 **멀쩡한 평균**을 반환

결측 비율 확인이 먼저

결측이 너무 많으면 남은 값으로 낸 통계는 신뢰 불가
계산 전에 결측 비율을 확인하는 습관 — **제외 고려**

nan 세 글자가 붙고 안 붙고로 결과가 완전히 달라짐

posco

nanmean · nanstd — 핵심 코드

결측을 무시한 평균 · 표준편차 — 일반 mean과 결과 비교

CODE

```
# 결측 포함 온도 배열
temp = np.array([70.1, np.nan, 71.2, 70.8])

# 일반 mean: 결측에 막혀 nan
print(np.mean(temp))
# 출력: nan

# nanmean: 결측 무시한 평균
print(round(float(np.nanmean(temp)), 2))
# 출력: 70.7
```

결측 처리의 핵심 원칙

결측은 '값이 0'이 아니라 '값을 모름'

주의 — 근거 없이 0으로 채우기

온도 결측을 0으로 채우면

0도가 실제 측정값처럼 취급되어
평균을 **크게 끌어내림**

모르는 것을 안다고 꾸미면 **분석 전체가 틀어짐**

✅ 세 가지 옵션

무시 · 추정값 · 제외

① **무시** — nanmean으로 건너뛰기
② **추정값 채우기** — 평균 · 중앙값
③ **제외** — 결측 많은 센서 빼기

이 원칙만 지켜도 결측 오류의 대부분을 피함

posco

실습 1. 결측 확인하고 결측 제외 통계 내기

빈 칸이 섞인 센서에서 결측 현황을 파악하고 나머지 값으로 통계 내기

목표

진동 센서의 결측 개수와 비율을 구하고, 결측을 뺀 평균을 계산

단계

① 유압 데이터 파일에서 진동 열 하나만 불러오기

② 값이 비어 있는 자리가 몇 개인지 세기

③ 그 개수를 전체 개수로 나눠 결측 비율 구하기

④ 빈 칸을 무시하고 나머지 값들의 평균 구하기

⑤ 빈 칸을 무시하지 않은 일반 평균도 함께 출력해 무엇이 다른지 확인

예상 결과

결측 6건 · 비율 0.12 · 결측 제외 평균 0.566 · 일반 평균은 nan

힌트 np.isnan(vib) → True 개수는 sum(), 비율은 mean() · 평균은 np.nanmean()

posco

백분위수 · 사분위수 — percentile · quantile

데이터를 크기순으로 줄 세웠을 때 특정 위치의 값

25 · 50 · 75 백분위수가 1사분위 · 중앙값 · 3사분위

평균과 달리 극단적인 이상값에 **휘둘리지 않음**

직관 — 100명을 키 순으로 세우면

25번째 사람의 키가 **25 백분위수**, 시험의 상위 10퍼센트가 **90 백분위수**
25 · 75 백분위수 사이에 전체 가운데 50% — 다음에 배울 **IQR의 핵심 재료**

평균이 다가 아님 — '상위 10퍼센트는 어디부터인가' 같은 질문에 답하는 도구

posco

percentile — 핵심 코드

사분위수 구하기 — percentile은 0~100, quantile은 0~1

CODE

```
# 진동 센서 6개 시점
vib = np.array([2.0, 2.2, 2.5, 2.8, 3.0, 6.5])

# 1사분위 (Q1)
print(round(float(np.percentile(vib, 25)), 2))
# 출력: 2.28

# 3사분위 (Q3)
print(round(float(np.percentile(vib, 75)), 2))
# 출력: 2.95
```

posco

사분위 기반 이상치 범위 — IQR

가운데 절반이 놓인 폭을 재고, 그 1.5배까지를 정상 울타리로 삼음

IQR — 가운데 절반이 놓인 폭을 재고, 그 1.5배까지를 울타리로



평균·표준편차와 달리 극단값에 흔들리지 않음 — 이상치가 섞인 데이터를 처음 훑을 때 먼저 쓰는 이유

값을 크기순으로 줄 세웠을 때 25% 자리가 Q1, 75% 자리가 Q3 — 그 사이 폭이 IQR

울타리를 벗어난 값만 이상치로 봄 — 평균·표준편차와 달리 극단값에 흔들리지 않음

1.5라는 배수는 관례로 굳은 값 — 더 민감하게 잡으려면 1.0, 느슨하게 보려면 3.0

posco

IQR — 핵심 코드

IQR 경계로 이상치 골라내기 — Q3 + 1.5×IQR을 넘는 값 추출

CODE

```
# Q1, Q3를 한 번에 구하기
q1, q3 = np.percentile(vib, [25, 75])
iqr = q3 - q1

# 상한 경계
upper = q3 + 1.5 * iqr

# 상한 초과 값만 골라내기
print(vib[vib > upper])
# 출력: [6.5]
```

조건 결합 시 각 조건을 괄호로 감싸기

posco

실습 2. IQR로 이상치 찾기

온도에서 가운데 절반의 폭을 구하고 울타리를 벗어난 값을 골라내기

목표

온도의 Q1·Q3로 상한을 계산하고 그 위의 값과 개수를 출력

단계

① 유압 데이터 파일에서 온도 열 하나만 불러오기

② 크기순 25% 자리와 75% 자리의 값을 한 번에 구하기

③ 두 값의 차이를 구하고, 위쪽 자리에 그 1.5배를 더해 상한 만들기

④ 상한을 넘는 온도 값만 골라내기

⑤ 골라낸 값과 개수를 출력해 실제로 몇 건이 걸리는지 확인

예상 결과

상한 37.5 · 걸리는 값 38.9 · 57.8 · 57.8 · 57.8 · 57.9 — 다섯 건

중간 점검 Q1=35.50 · Q3=36.30 · IQR=0.80 | 힌트 np.nanpercentile(t, [25,75])

평균·표준편차로 표준화 — z-score

각 값이 평균에서 표준편차의 몇 배 떨어졌는지를 나타내는 숫자



단위가 다른 센서도 표준화하면 같은 눈금 위에 놓여 나란히 비교할 수 있음

실습에서는 절댓값 2를 기준으로 이상 후보를 표시 — 부호는 위로 벗어났는지 아래로 벗어났는지

이 데이터의 온도 최대 z-score는 3.39 — 평균에서 표준편차의 3.4배만큼 떨어져 있다는 뜻

posco

z-score — 핵심 코드

z-score로 표준화하고 절댓값 2 초과 이상치 추출

CODE

```
# z-score = (값 - 평균) / 표준편차
z = (vib - np.mean(vib)) / np.std(vib)

# 절댓값 2 초과 = 이상치
print(vib[np.abs(z) > 2])
# 출력: [6.5]
```

posco

실습 3. z-score로 이상 시점 찾기

온도를 표준화해 평균에서 크게 벗어난 사이클을 찾기

목표

온도의 z-score 절댓값이 2를 넘는 사이클 번호를 출력

단계

① 유압 데이터 파일에서 시점과 온도 두 열을 불러오기

② 온도의 평균과 흩어진 정도를 구하기

③ 각 값에서 평균을 빼고 흩어진 정도로 나눠 표준화하기

④ 방향을 무시하도록 부호를 없앤 뒤 2를 넘는 자리를 골라내기

⑤ 그 자리의 사이클 번호를 출력해 몇 번째 사이클이 걸리는지 확인

예상 결과

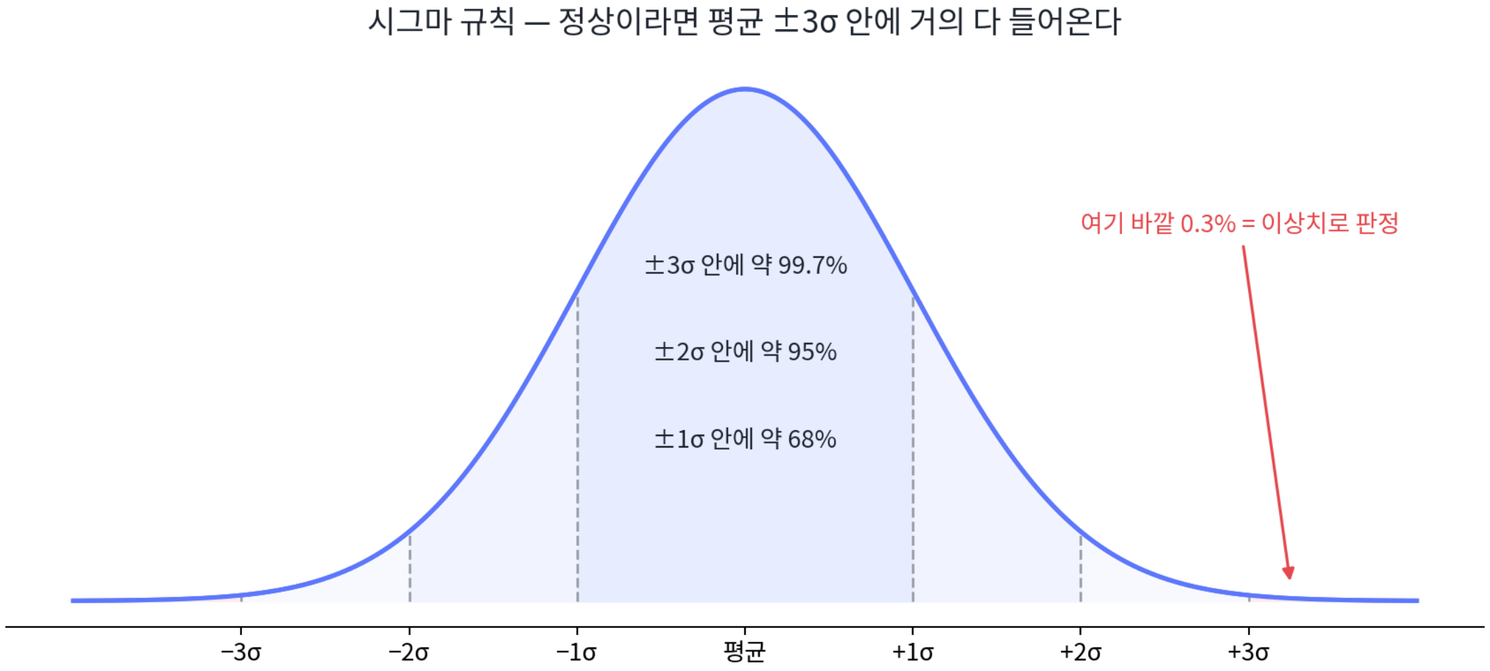
47 · 48 · 49 · 50 네 사이클이 출력 — 최대 z-score는 3.39

힌트 z=(t-평균)/표준편차 · np.abs(z)>2 · 인덱스가 아닌 cyc의 값을 출력

posco

정상 범위 정의 — 시그마 규칙

평균을 중심으로 표준편차 일정 배수 안쪽을 정상으로 정의



값이 종 모양으로 퍼져 있다면 평균 ±3σ 안에 전체의 99.7%가 들어옴

그 바깥 0.3%를 이상으로 보는 것이 시그마 규칙 — z-score의 임계값 3과 같은 말

종 모양이 아닌 데이터에는 비율이 맞지 않음 — 한쪽으로 치우친 데이터에는 IQR이 안전

posco

시그마 규칙 — 핵심 코드

평균 ± 3시그마로 정상 범위 — 상한 초과 시점만 골라내기

CODE

```
# 평균과 표준편차
m, s = np.mean(vib), np.std(vib)

# 3시그마 상한
upper = m + 3 * s

# 상한을 넘는 값 찾기
print(vib[vib > upper])
# 출력: []
```

작은 데이터에서 3시그마는 느슨, 핵심 설비면 2시그마로 엄격

posco

실습 4. 3시그마 경보 판정하기

평균에서 표준편차 세 배만큼 떨어진 자리를 기준선으로 삼아 경보 내기

목표

온도의 3시그마 상한을 구하고 그 위에 있는 사이클 번호를 출력

단계

① 유압 데이터 파일에서 시점과 온도 두 열을 불러오기

② 온도의 평균과 흩어진 정도를 구하기

③ 평균에 흩어진 정도의 세 배를 더해 상한 만들기

④ 상한을 넘는 자리의 사이클 번호를 골라내기

⑤ 상한값과 걸린 사이클 번호를 함께 출력해 실습 3 결과와 비교

예상 결과

상한 55.58 · 걸리는 사이클은 47 · 48 · 49 · 50 — 실습 3과 같은 네 건

힌트 상한=평균+3×표준편차 · 비교는 t>상한 · 실습 3과 임계값 차이 확인

posco

IQR vs z-score 비교표

데이터 분포와 이상값 영향에 따라 두 기법의 적합도가 갈림

구분	IQR	z-score
기준	사분위수 (Q1, Q3)	평균 · 표준편차
적합 분포	치우친 분포	정규분포 (종 모양)
이상값 영향	거의 없음 (안정적)	받음 (휘둘림)
해석	보통 (Q1·Q3 이해 필요)	쉬움 (표준편차 몇 배)

치우친 분포 → IQR · 깔끔한 종 모양 → z-score. 이상값 많을수록 IQR이 버팀

posco

02

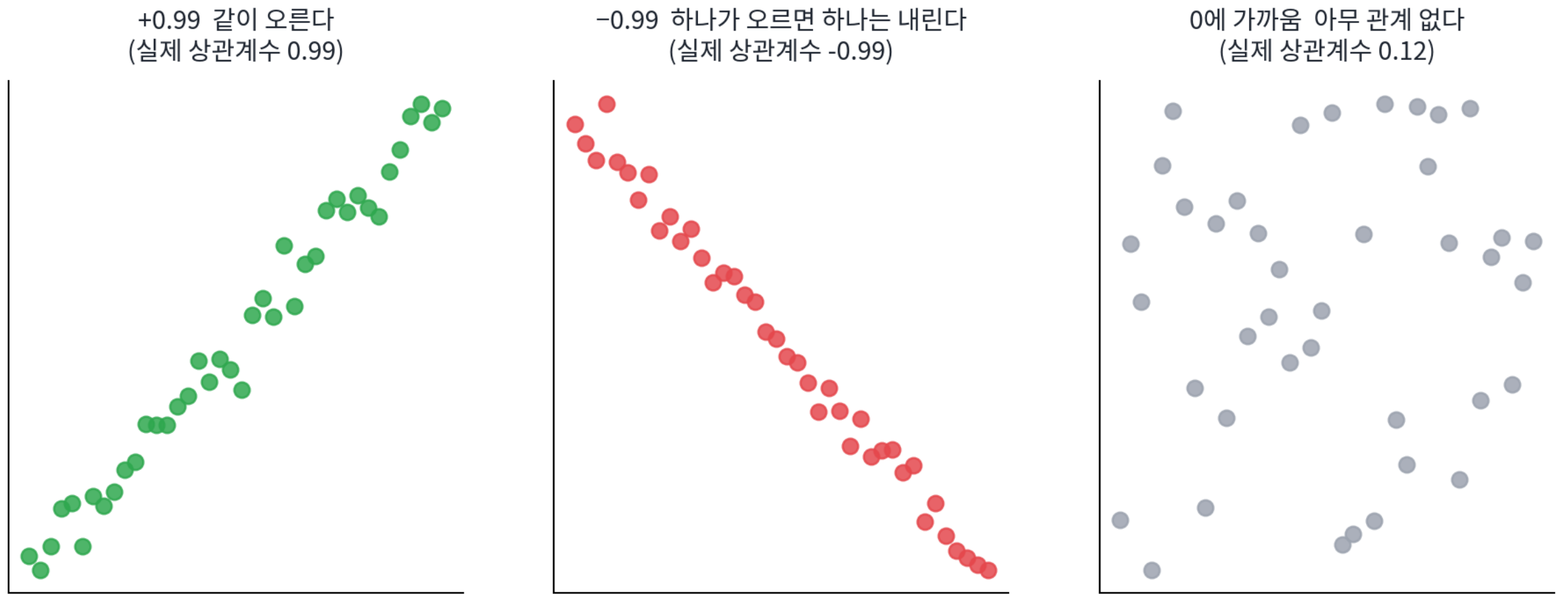
다중 센서와 종합 분석

corrcoef · axis별 통계 · 변화점 탐지 · 종합 이상 점수

센서 간 상관관계 — corrcoef

두 센서가 얼마나 함께 움직이는지를 하나의 숫자로 표현

상관계수 — 부호는 방향, 절댓값은 세기. −0.99는 '관계가 약하다'가 아니라 '정확히 반대로 강하게 움직인다'



부호는 방향, 절댓값은 세기 — 음수는 관계가 약한 것이 아니라 반대로 움직인다는 뜻

상관은 함께 변하는 패턴을 보여줌 · 단독으로 고장 원인이나 인과를 증명하지는 못함

전체 합성 데이터에서 온도·냉각효율의 상관은 약 −0.99 · 같은 양만큼 변한다는 뜻은 아님

posco

corrcoef — 핵심 코드

두 센서의 상관계수 — 0.998이면 강하게 함께 움직임

CODE

```
# 온도와 진동 두 센서
temp = np.array([70, 72, 74, 76, 78])
vib = np.array([2.0, 2.2, 2.5, 2.7, 3.0])

# 상관행렬의 [0,1] 칸이 두 센서의 상관계수
print(round(float(np.corrcoef(temp, vib)[0, 1]), 3))
# 출력: 0.998
```

부호가 방향, 절댓값이 세기 — 실습에서 만날 온도와 냉각효율은 −0.99

실습 5. 센서 간 상관관계 확인하기

온도와 냉각효율이 함께 움직이는지 숫자 하나로 확인

목표

두 센서의 상관계수를 구하고 부호와 크기가 무엇을 뜻하는지 해석

단계

① 유압 데이터 파일에서 온도와 냉각효율 두 열을 불러오기

② 두 묶음의 상관계수를 구하기

③ 결과가 표 모양으로 나오므로 필요한 칸 하나만 꺼내기

④ 소수점 둘째 자리까지 다듬어 출력

⑤ 부호와 크기를 보고 두 센서가 어떤 관계인지 말로 설명해 보기

예상 결과

−0.99 — 온도가 오르면 냉각효율이 거의 정확히 그만큼 떨어짐

힌트 np.corrcoef(t, ce)의 결과는 2×2 · 필요한 칸은 [0,1] · 부호=방향

posco

2D 배열 axis별 통계 활용

행은 센서, 열은 시점인 행렬에서 axis로 방향을 정해 통계

01 · axis=1

각 행을 옆으로 — 센서별 통계

센서마다의 정상 수준

열 방향이 합쳐져 사라지고 센서별 결과만 남음. 센서마다 평균 · 표준편차가 따로 산출됨

```
np.mean(data, axis=1)
```

02 · axis=0

각 열을 아래로 — 시점별 통계

시점마다의 전체 상태

행 방향이 합쳐져 사라지고 시점별 결과만 남음. 종합 점수 합산에 활용

```
np.mean(data, axis=0)
```

axis는 **없어지는 방향** — axis=1을 주면 열 방향이 합쳐져 센서별 결과만 남음

posco

axis별 통계 — 핵심 코드

센서별 평균과 시점별 평균 — axis=1은 센서 수만큼, axis=0은 시점 수만큼

CODE

```
# 행은 센서, 열은 시점인 2D 행렬
data = np.array([[70., 72., 74.],
 [2.0, 2.2, 2.5]])

# axis=1 → 센서별 평균 (행 방향)
print(np.round(np.mean(data, axis=1), 2))
# 출력: [72. 2.23]
```

axis=0은 시점별 평균 3개, axis 생략 시 전체 평균 하나

실습 6. 센서별 통계 한 번에 구하기

네 센서를 표로 쌓고 방향을 정해 센서마다 따로 통계 내기

목표

행이 센서인 표를 만들어 센서별 평균과 흩어진 정도를 한 번에 계산

단계

① 유압 데이터 파일에서 온도·진동·냉각효율·냉각출력 네 열을 불러오기

② 네 묶음을 위아래로 쌓아 행이 센서, 열이 시점인 표 만들기

③ 표에서 센서마다 따로 평균이 나오도록 방향을 지정하기

④ 같은 방향으로 흩어진 정도도 구하기

⑤ 두 결과를 소수점 첫째 자리까지 다듬어 출력하고 값이 네 개씩 나오는지 확인

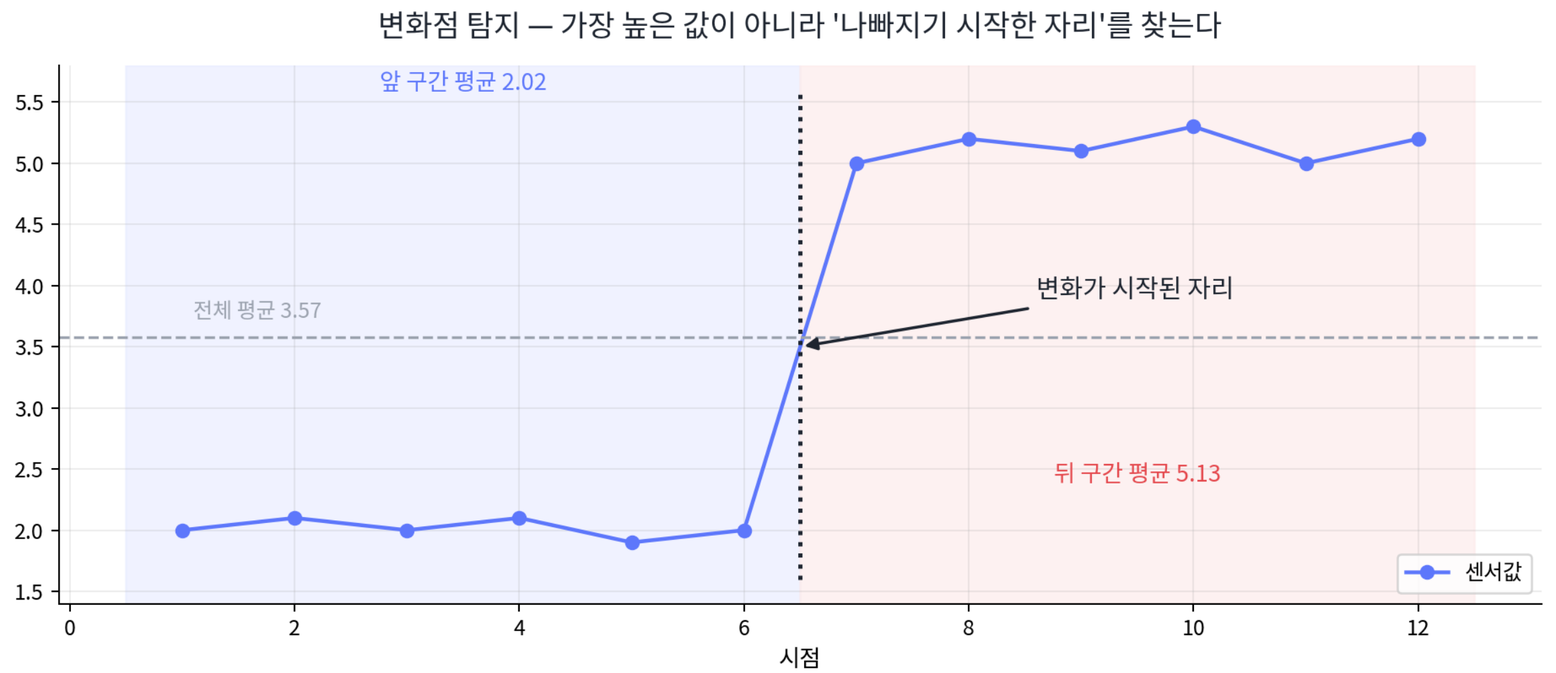
예상 결과

센서별 평균 37.7 · 0.6 · 44.7 · 57.7, 흩어진 정도 6.0 · 0.1 · 7.6 · 1.6

중간 점검 M.shape=(4,50) | 힌트 np.stack, np.nanmean(...,axis=1)

통계량 급변 구간 — 변화점 탐지

전체 데이터를 이용해 상태가 달라지는 경계 후보를 사후에 추정



앞뒤 구간의 평균이 크게 갈리는 자리가 상태가 달라진 지점

평균을 뺀 값을 누적하면 평균보다 낮은 구간에서는 내려가고 높은 구간에서는 올라감

누적 합 최솟값은 상승 구간 직전의 경계 후보 · 실시간 조기 경보와는 구별

변화점 탐지 — 핵심 코드

누적 합이 가장 낮아지는 자리가 변화 직전 — 반복문 없이 두 줄로

CODE

```
# 앞은 안정, 뒤는 상승한 진동 데이터
vib = np.array([2.0, 2.1, 2.0, 5.0, 5.2, 5.1])

# 평균 뺀 값의 누적 합
cs = np.cumsum(vib - np.mean(vib))

# 가장 낮은 위치 = 변화 직전
print(np.argmin(cs))
# 출력: 2
```

평균을 뺀 값을 누적하면, 평균보다 낮은 구간에서는 내려가고 높은 구간에서는 올라감

실습 7. 변화 경계 후보 찾기

평균 편차의 누적 합으로 온도 수준이 달라진 경계 후보를 찾기

목표

누적 합 최솟값 위치를 찾아 다음 사이클과의 경계로 해석

단계

① 유압 데이터 파일에서 시점과 온도 두 열을 불러오기

② 온도의 전체 평균을 구하기

③ 각 값에서 평균을 뺀 뒤 앞에서부터 누적해 더하기

④ 누적한 값이 가장 작아지는 자리를 찾기

⑤ 그 자리의 사이클 번호를 출력해 실습 3·4가 찾은 시점과 비교

예상 결과

45번 사이클 뒤(45~46번 사이)가 경계 후보 · 47번은 경보 초과 시작

중간 점검 len(cs)=50 | 힌트 np.cumsum(t-np.nanmean(t)), np.argmin(cs)

posco

여러 센서 종합 이상 점수화

두 센서의 표준화 편차를 더해 시점별 합성 이상 점수 생성

각 센서를 z-score 절댓값으로 바꿔 같은 단위로 합산

한 센서만 튀면 점수 소폭, **여러 센서 동시면 점수 급등**

의사의 진단처럼 — 여러 지표 종합

의사는 체온 하나만 보지 않고 혈압 · 맥박을 종합 — 설비도 마찬가지
z-score 표준화로 단위 다른 센서들도 **더할 수 있게 됨**

여러 센서 신호를 함께 살피는 예시 · 점수 자체가 고장 확률은 아님

종합 이상 점수 — 핵심 코드

센서별 z-score 절댓값 합산 — axis=0으로 시점별 점수

CODE

```
# 온도(1열)와 냉각효율(3열)로 2×50 배열 생성
F = "data/11_hydraulic_cooler.csv"
data = np.genfromtxt(F, delimiter=",", skip_header=1, usecols=(1, 3)).T
m = np.nanmean(data, axis=1, keepdims=True)
s = np.nanstd(data, axis=1, keepdims=True)
score = np.abs((data - m) / s).sum(axis=0)
print(np.argmax(score))      # 49 (0부터 세는 인덱스)
print(np.argmax(score) + 1)  # 50 (사이클 번호)
```

keepdims=True는 센서별 통계 모양을 (2,1)로 유지해 브로드캐스팅에 도움

posco

03

종합 이상 탐지 리포트

오늘 배운 기법을 하나로 엮어 정비 판단까지

실습 8. 종합 이상 점수로 정비 우선순위 정하기

여러 센서의 이상 신호를 하나의 점수로 합쳐 이상 점수가 가장 높은 시점을 가려내기

목표

센서별 z-score 절댓값을 합쳐 시점별 점수를 만들고 상태 기록과 대조

단계

① 유압 데이터 파일에서 시점·온도·냉각효율·상태 네 열을 불러오기

② 온도와 냉각효율을 각각 표준화하고 부호를 없애기

③ 두 값을 시점마다 더해 종합 점수를 만들기

④ 점수가 가장 높은 시점을 찾고, 종합 점수가 큰 순서로 상위 세 시점도 뽑기

⑤ 찾아낸 시점이 교육용 상태 라벨(47~50번)과 맞는지 대조해 출력

예상 결과

최고 점수 50번(6.79) · 상위 세 시점 50 · 49 · 48 · 교육용 상태 라벨(47~50번)은 47~50

중간 점검 score.shape=(50,) | 힌트 np.argsort(score)[::-1][:3] · 상태 라벨 대조

posco

전체 이상 탐지 5단계

준비 → 파악 → 탐지 → 종합 → 보고

STEP 1

준비

데이터 불러와 결측 처리

`isnan · nanmean`

→

STEP 2

파악

평균 · 표준편차 · 사분위수로 정상 수준 파악

`mean · std · percentile`

→

STEP 3

탐지

IQR · z-score로 이상치 찾기

`boolean · where`

→

STEP 4

종합

종합 이상 점수의 높은 시점 · 변화 경계 후보 확인

`sum · argmax · cumsum`

→

STEP 5

보고

결과를 사람이 읽기 좋게 정리

정비 판단의 마침표

NumPy를 넘어 어떤 도구를 쓰든 통용되는 보편적인 흐름

CLOSING

posco

분석은 질문에 답해가는 과정

데이터 분석은 도구의 나열이 아니라 **질문에 답해가는 과정**

QUESTION 01 · 정상

정상은 어느 수준인가, 벗어난 값은 무엇인가

QUESTION 02 · 시점

어느 시점의 점수가 높았나, 변화 경계 후보는 어디인가

QUESTION 03 · 행동

그래서 무엇을 해야 하는가

세 질문에 차례로 답하다 보면 오늘 배운 도구가 자연스럽게 제자리를 찾아감

CLOSING

posco

NumPy에서 Pandas로 — 다음 단계

Pandas는 표를 다루는 도구이며 NumPy 위에 지어짐

실무 데이터는 보통 엑셀 같은 표 형태

오늘 배운 통계 · 필터링 원리가 Pandas에서도 **그대로 통용**

NumPy는 골조, Pandas는 사무실

표 안에서 실제 숫자 계산을 하는 것은 결국 NumPy 배열
기초를 단단히 한 사람이 결국 **멀리 감**

오늘로 NumPy 실습은 거의 끝 — 다음은 열 이름 · 날짜가 붙은 표를 다루는 Pandas

posco

NumPy vs Pandas 비교표

오늘 배운 원리가 Pandas에서도 그대로 통용

구분	NumPy	Pandas
다루는 것	숫자 배열	이름 붙은 표
강점	빠른 계산	표 정리 · 필터
관계	기초 토대	위에 구축

Pandas는 NumPy 위에 지어졌음. 오늘 배운 게 헛되지 않음

APPENDIX · 수업자료 사용 안내

유압 냉각기 분석 — 교육용 합성 데이터 안내

데이터 파일: data/11_hydraulic_cooler.csv | 50사이클 × 6열

01

데이터 형태

- cycle · 1~50 사이클 번호
- oil_temperature · 오일 온도
- vibration · 진동(결측 6건)

02

분석 지표

- cooling_efficiency · 냉각효율
- cooling_power · 냉각출력
- status · 0 정상 / 1 이상

03

실습 검산 기준

- IQR 상한 37.50 · 이상 후보 5건
- 3시그마 상한 55.58 · 47~50번
- 종합 점수 최고 50번 · 약 6.79

중요 본 파일은 실습 조건에 맞춰 새로 생성한 합성 데이터입니다.

UCI 원본에서 수집한 실제 센서 측정치가 아니며, 상태 라벨·통계 점수는 실제 고장 판정이나 정비 기준으로 사용할 수 없습니다.

CSV는 강사용 Python 파일과 같은 폴더의 data 디렉터리에 두고 실행하세요.